Open in app ↗ Sign up Sign in

Medium Search Write

PAGE NOT FOUND

404

Out of nothing, something.

You can find (just about) anything on Medium — apparently even a page that doesn't exist. Maybe these stories will take you somewhere new?

Home

Open in app ↗ Sign up Sign in

Medium Search Write

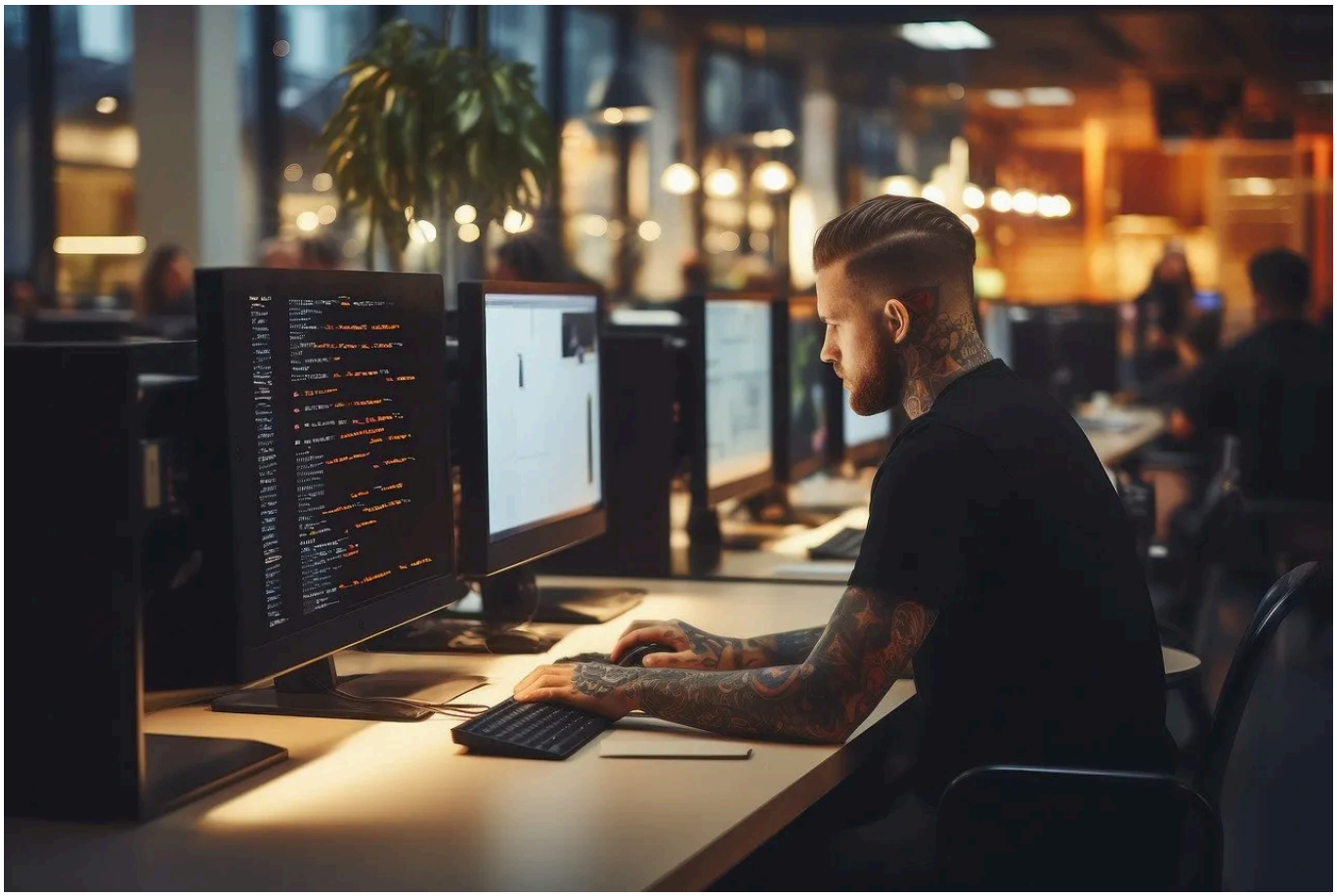
Router + Expressjs + Node Js Tutorial + Mern Stack + Web Development +

Understanding Router in Express.js: A Complete Guide



Aryan kumar Follow 5 min read · Oct 4, 2024

90 5



When building scalable web applications, organizing your routes efficiently is key. Express.js, a minimal and flexible Node.js framework, provides an easy way to manage routing using its `Router` class. If you've worked with simple route definitions in Express before, using `express.Router()` will take your application to the next level in terms of structure and maintainability.

In this article, we'll explore what an Express Router is, why you should use it, and how you can implement it in your applications.

What is the Express Router?

The **Router** in Express.js is a mini Express application without a complete set of middleware like the main app object (`app`). It provides a way to create

modular and mountable route handlers. This allows you to keep your routes organized and manageable, especially in large applications.

While in smaller applications you may define routes directly on the `app` object, in larger projects, using a Router helps break down the code into smaller, reusable components.

Simple Explanation:

Think of a **Router** as a mini Express app that you can attach routes to and then plug into your main Express app. This way, you can modularize routes based on features, resource types, or sections of your app.

Why Use an Express Router?

Using a router has several benefits:

1. **Code Organization**: It allows you to split your routes across different files or modules, making your code easier to manage.
2. **Reusability**: You can reuse route modules across different parts of your application or even different projects.
3. **Scalability**: As your application grows, managing everything in one file becomes hard. Routers make scaling your codebase simpler.

Setting Up and Using the Router

Let’s go step by step to set up and use an Express Router.

Step 1: Install Express

First, make sure you have Express installed:

```
npm install express
```

Step 2: Create a Simple Express App

Let's start with a basic Express app. Create an `app.js` file:

```
const express = require('express');
const app = express();
const port = 3000;

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

Step 3: Defining Routes Without a Router

If you're not using a router, you would define routes directly on the `app` object like this:

```
app.get('/', (req, res) => {
  res.send('Home Page');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});
```

While this is fine for smaller applications, as your app grows, this can quickly become overwhelming. Here's where the Router comes in.

Step 4: Using `express.Router()`

Creating a Router

Let's refactor the previous example by creating a **Router** for the `/about` route.

1. **Create a separate router file** (e.g., `aboutRoutes.js`):

```
const express = require('express');
const router = express.Router();

// Define the routes for this router
router.get('/', (req, res) => {
  res.send('About Page');
});

module.exports = router;
```

In your main `app.js`, you'll now import the router and tell your main app to use it:

```
const express = require('express');
const aboutRoutes = require('./aboutRoutes'); // Import the router
```

```
const app = express();
const port = 3000;

// Use the router for the /about path
app.use('/about', aboutRoutes);

app.get('/', (req, res) => {
  res.send('Home Page');
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

Now, the `/about` route is handled by the **`aboutRoutes`** router. If you visit `http://localhost:3000/about`, you'll see the response from the router.

Step 5: Organizing Routes with Multiple Routers

One of the great things about Express Router is that you can create multiple routers, each handling a different section of your app.

Example: Adding a Users Router

1. **Create a new router file** for the `/users` routes (e.g., `usersRoutes.js`):

```
const express = require('express');
const router = express.Router();

// Define routes for users
router.get('/', (req, res) => {
  res.send('Users Home Page');
});

router.get('/:id', (req, res) => {
```

```
  const userId = req.params.id;
  res.send(`User ID: ${userId}`);
});

module.exports = router;
```

Update your `app.js` to use both routers:

```
const express = require('express');
const aboutRoutes = require('./aboutRoutes');
const usersRoutes = require('./usersRoutes');
const app = express();
const port = 3000;

app.use('/about', aboutRoutes);  // Use the /about router
app.use('/users', usersRoutes);  // Use the /users router

app.get('/', (req, res) => {
  res.send('Home Page');
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

In this example, we have two routers: one for the `/about` path and one for the `/users` path. This modular approach keeps your codebase cleaner and easier to maintain.

Middleware with Routers

Express Router supports middleware, allowing you to run custom logic for specific routes or groups of routes.

Applying Middleware to a Router

For example, let's add a logging middleware that logs the request URL and time for all routes within a router.

1. **Modify your `aboutRoutes.js` file:**

```
const express = require('express');
const router = express.Router();

// Middleware that logs the request details
router.use((req, res, next) => {
  console.log(`Request URL: ${req.originalUrl}, Time: ${new Date().toISOString()}
  next();
});

router.get('/', (req, res) => {
  res.send('About Page');
});

module.exports = router;
```

Now, every time a request hits any of the routes within the `/about` path, the middleware will log the request URL and time.

Route Parameters with Routers

You can use **route parameters** in routers just like in regular routes. Route parameters allow you to capture dynamic segments of the URL.

Get Aryan kumar's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

For example, in our `usersRoutes.js`, we already defined a parameterized route:

```
router.get('/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID: ${userId}`);
});
```

In this case, `:id` is a route parameter. So, if a user visits `/users/123`, the app will respond with `User ID: 123`.

Nesting Routers

One of the most powerful features of Express Router is that you can **nest routers**. This allows you to create complex routing structures while keeping everything modular.

Example of Nested Routers:

Let's say you want to create an API structure where users have posts. You can nest the `/posts` router inside the `/users` router.

1. **Create the posts router** (e.g., `postsRoutes.js`):

```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('List of user posts');
});

router.get('/:postId', (req, res) => {
  const { postId } = req.params;
  res.send(`Post ID: ${postId}`);
});

module.exports = router;
```

Modify your `usersRoutes.js` to include the posts router:

```
const express = require('express');
const router = express.Router();
const postsRoutes = require('./postsRoutes');

router.get('/', (req, res) => {
  res.send('Users Home Page');
});

router.get('/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});

// Nest the posts router
router.use('/:id/posts', postsRoutes);

module.exports = router;
```

In this setup, if you visit `/users/123/posts`, the `postsRoutes` router will handle the request and return the list of posts for the user. This nesting capability keeps things intuitive and scalable as you add more features.

If you found value in this post, don't forget to follow me for more insights on web development, programming tips, and the latest in tech! Your support really helps — please clap for this article if you enjoyed it.

Also, stay connected:

Follow me on Twitter for quick coding tips and updates.
Check out my YouTube Channel for deep dives into development projects and tutorials.
Join my learning journey on Udemy and master full-stack development, TypeScript, and more!
Let's level up our skills together! 👨‍💻

Router + Expressjs + Node Js Tutorial + Mern Stack + Web Development +



Written by Aryan kumar

197 followers · 69 following

Follow

Responses (5)

Write a response

What are your thoughts?

Ramanathasan Mariyanesan he/him
Mar 22, 2025

A comprehensive guide to understanding routers in Express.js! The article clearly explains how to set up and use routers for modular routing, providing useful examples to help developers build scalable and maintainable Express applications.

48 Reply

Lakshmikanth Mhetre
Feb 10

A quick 5 min read, refreshed all the content.

Reply

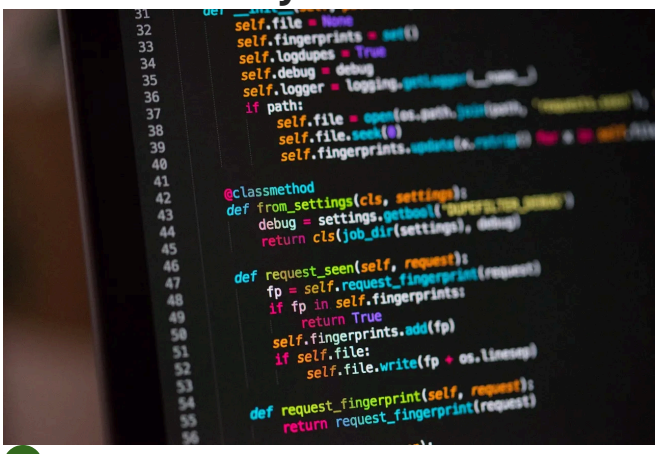
nirmitee
Jan 18

Great Read !!!

Reply

See all responses

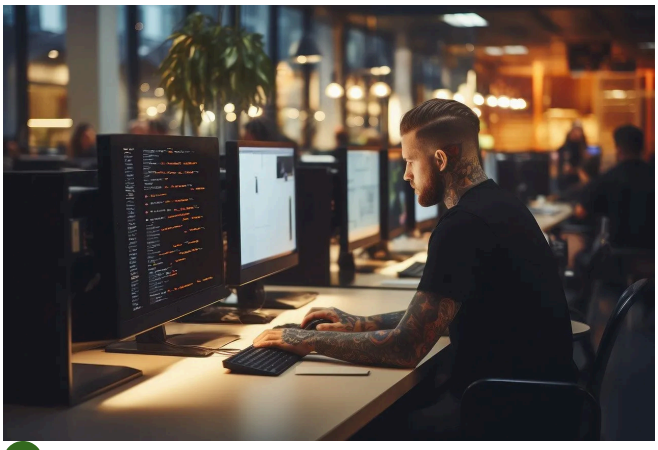
More from Aryan kumar



Aryan kumar · Oct 4, 2024

Mastering Route Parameters and Query Strings in Express.js

In a web application, dynamic routing plays a vital role in managing URLs. Express.js makes it incredibly easy to capture values from URLs...

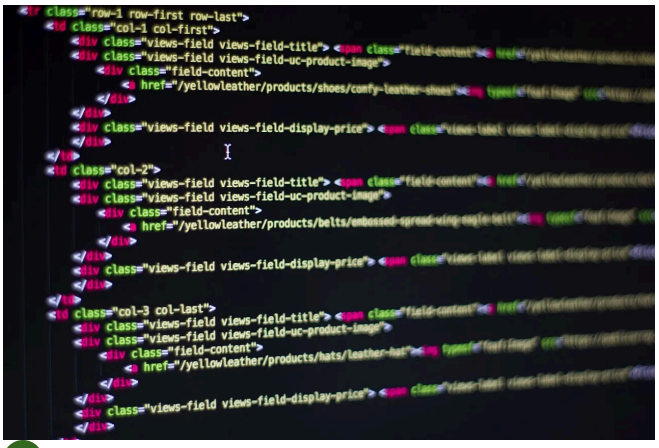


Aryan kumar · Oct 4, 2024

Mastering Express.js Controllers: The Key to Clean and Scalable Applications

When building scalable applications, it's essential to follow clean architecture patterns. In Express.js, controllers are a powerful...

51 1



Aryan kumar · Sep 13, 2024

Array Grouping in JavaScript: A Quick and Efficient Guide

Arrays in JavaScript are extremely versatile, but there are times when we need to organize or group elements based on specific criteria...

3 3

89 2



Aryan kumar · Sep 29, 2024

Handling Images and Assets in React.js

When building web applications with React, working with images and other assets (like fonts or videos) is something you'll do often...

9

See all from Aryan kumar